

4. Functions and Reading Inputs from File

4.1	Motivating application: Cooling of a nuclear fuel rod	42
4.2	Reading inputs from file	44
4.3	Functions	48
4.4	Homework	51

Introduction

This chapter extends the solver developed in Chapter 3 to simulate steady heat conduction with a spatially varying thermal conductivity, $k(x, y)$. In the process, we introduce how to read inputs from file, and offload repetitive tasks to basic C++ functions. The end-of-chapter homework applies the code to simulate the temperature distribution in a nuclear fuel rod, and explores the accuracy of the simulated results by performing a grid convergence study.

4.1 Motivating application: Cooling of a nuclear fuel rod

This chapter extends our finite-volume code to permit the simulation of steady heat conduction problems for which the thermal conductivity varies spatially, $k(x, y)$. We then use the code to simulate the steady temperature distribution in a nuclear fuel rod immersed in a liquid coolant.¹ We consider the 2D domain $x \in [0, L_x]$ and $y \in [0, L_y]$ sketched in Fig. 4.1, with the rod axis located at $(x_c, y_c) = 0.5(L_x, L_y)$. The rod consists of an inner fuel core of radius R_1 and thermal conductivity k_1 , surrounded by a cladding of outer radius R_2 and thermal conductivity k_2 . The remaining domain is a liquid coolant of thermal conductivity k_3 .

Within the fuel core, nuclear fission produces a volumetric heat source $s(x, y)$ that varies as

$$s = a \left[1 + b \left(\frac{r}{R_1} \right)^2 \right], \quad r = \sqrt{(x - x_c)^2 + (y - y_c)^2}, \quad (4.1)$$

where r is the radial distance from the rod center. The volumetric heat generation varies from a minimum $s = a$ at $r = 0$ to a maximum $s = a(1 + b)$ at $r = R_1$. There is no heat generation in the outer cladding. To focus on steady heat conduction, we neglect radiative heat transfer and assume the coolant is at rest. We model the cooling effect of the coolant by applying a constant temperature $T = T_c$ on the four edges of the computational domain.

Our engineering objectives are to compute the maximum temperature T_{max} in the fuel core, and the total heat transfer rate $\dot{Q}$ (units W/m) generated by the fuel core per unit depth in the z -direction. Because this is a steady heat conduction problem, $\dot{Q}$ can be evaluated by computing the total heat transfer rate exiting the domain boundary through the coolant. All physical parameters are listed in Table 4.1.

An important modeling decision in this problem is how to handle the interfaces between the three materials. One approach is to build a grid that conforms to the material interfaces at $r = R_1$ and $r = R_2$. Using a structured grid, this could be achieved by transforming to a cylindrical coordinate system, as illustrated by the finite-volume grid shown in Fig. 4.2. At each interface, we would then enforce continuity of temperature and heat flux. For example, at the interface $r = R_1$ between the fuel core and cladding, these conditions read

$$T_1 \Big|_{r=R_1} = T_2 \Big|_{r=R_1}, \quad k_1 \frac{\partial T_1}{\partial r} \Big|_{r=R_1} = k_2 \frac{\partial T_2}{\partial r} \Big|_{r=R_1}, \quad (4.2)$$

where T_1 and T_2 denote the temperature fields in the fuel core and cladding, respectively.

In this chapter, however, we adopt a simpler approach that discretizes the full domain using a uniform Cartesian grid, and treats the thermal conductivity as a spatially varying

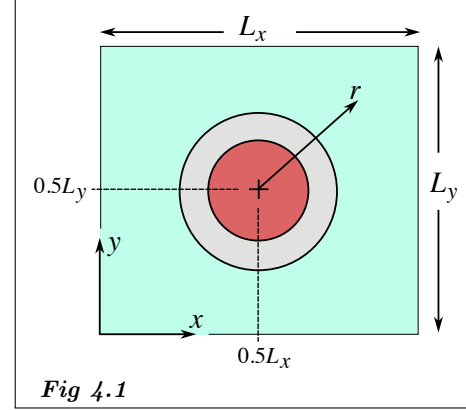


Fig 4.1

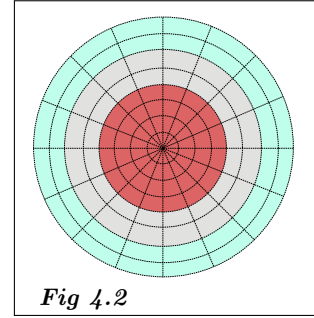


Fig 4.2

¹This problem is adapted from Section 10.3 of a classic textbook by Bird, Stewart, and Lightfoot [1], which considers heat conduction in a spherical nuclear fuel element.

L_x	15 mm	k_1	2.0 W/m/K
L_y	15 mm	k_2	15.0 W/m/K
R_1	4 mm	k_3	0.6 W/m/K
R_2	5 mm	a	10^7 W/m ³
T_c	300 K	b	1.0

Table 4.1: Physical parameters.

function,

$$k(x, y) = \begin{cases} k_1, & r \leq R_1, \\ k_2, & R_1 < r \leq R_2, \\ k_3, & r > R_2. \end{cases} \quad (4.3)$$

This simplicity comes at the cost of degrading the spatial accuracy of the simulated fuel rod, because the discontinuous thermal conductivity violates the smoothness assumption underlying our finite-volume approximation. We explore the extent of this degradation in the end-of-chapter homework.

4.2 Reading inputs from file

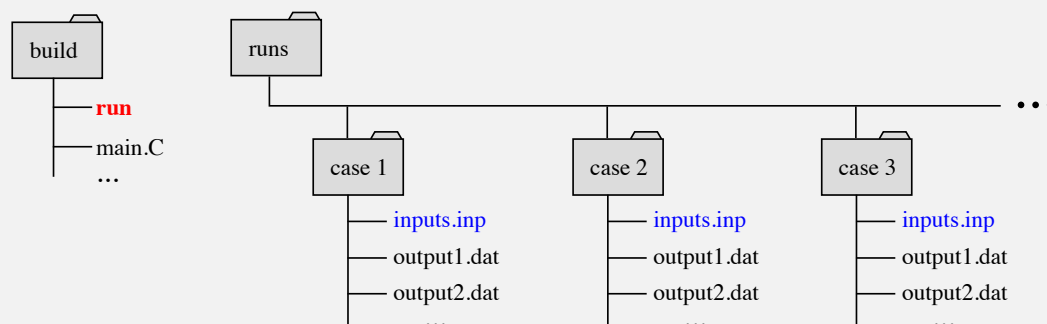
When a simulation requires many input parameters, it becomes more convenient to read the parameters from an input file than from the terminal. This allows the user to modify the input file and rerun the program without recompiling. In this course, we assume that input files are formatted as in File (4.1) below, where each line contains a numeric input followed by a semicolon, and an optional comment describing the input. Though not required, we also add the file extension `.inp` for all input files.

File 4.1: A file named `inputs.inp` with four input parameters.

```
1 16; Nx (number of cells in x)
2 32; Ny (number of cells in y)
3 2.1; Lx (domain length in x)
4 3.2; Ly (domain length in y)
```

Note: Organizing simulation files

When running a suite of simulations for many different combinations of input parameters, a standard practice is to store the source files and compiled executable in a single build directory, as sketched below. Meanwhile, the input files for each case are stored in separate dedicated directories. Though beyond the scope of this course, the entire process of calling the executable for each case and running any associated post-processing routines can be automated using a Python script (or shell script).



4.2.1 Opening a file and reading a line of text

We begin with the example in File (4.2) below, which demonstrates how to open a file and extract a line of text.

File 4.2: Opening a file and reading individual lines.

```
1 #include <fstream> // needed for reading files
2 #include <iostream> // needed for cout
3 #include <string> // needed for string
4
5 int main()
6 {
7     // define a string
```

```

8     std::string line;
9
10    // Open the input file
11    std::ifstream infile;
12    infile.open("inputs.inp");
13
14    // Read a single line and display in terminal
15    std::getline(infile, line);
16    std::cout << "line = " << line << std::endl;
17
18    // Repeat for next line
19    std::getline(infile, line);
20    std::cout << "line = " << line << std::endl;
21
22    // Close the file
23    infile.close();
24
25    return 0;
26}

```

File 4.2 performs the following operations:

- Line 8 defines a variable named `line` of type `string`:

```
8     std::string line;
```

A `string` is a data type that stores a sequence of characters, such as words or sentences.

- Lines 11–12 prepare the program to read data from a file named `inputs.inp`:

```
11    std::ifstream infile;
12    infile.open("inputs.inp");
```

These commands do two things. First, line 11 creates a variable named `infile` of type `ifstream`, which is a data type used for reading files. Then line 12 opens the file called `inputs.inp`, allowing the program to access its contents line by line.

- The operations on lines 11–12 require the `<fstream>` header included on line 1 of the file.
- Line 15 reads the first line of the file `inputs.inp` and stores it in the variable `line`:

```
15    std::getline(infile, line);
```

The command `std::getline` reads all characters from the current position in the file until it reaches a newline character and stores them in `line`. When the file is first opened, the current position is at the beginning of the file. Each time `getline` is called, the current position moves to the start of the next line.

- Line 16 displays the string stored in `line` to the terminal:

```
16    std::cout << "line = " << line << std::endl;
```

This demonstrates that `cout` can display variables of type `string`, just as it can display integers or doubles.

- Lines 19–20 repeat the process to read and display the next line of the file. Line 23 then closes the file.

Compiling and running File 4.2 generates the terminal output shown below:

```
line = 16;  Nx (number of cells in x)
line = 32;  Ny (number of cells in y)
```

Notice that each line is printed exactly as it appears in the input file.

4.2.2 Extracting the numerical input

The next example, shown in File (4.3) below, demonstrates how to extract the numerical data preceding the semicolon, and assign it to the desired variable.

File 4.3: Extracting numerical data and assigning to variable.

```
1 #include <fstream>    // needed for reading files
2 #include <iostream>   // needed for cout
3 #include <sstream>    // needed for stringstream
4 #include <string>     // needed for string
5
6 int main()
7 {
8     int Nx, Ny;
9     double Lx, Ly;
10    std::string line;
11
12    // open file
13    std::ifstream infile;
14    infile.open("inputs.inp");
15
16    // --- Read first line ---
17    std::getline(infile, line); // readline
18    std::stringstream(line) >> Nx; // extract number before ";"
19
20    // --- Read second line ---
21    std::getline(infile, line);
22    std::stringstream(line) >> Ny;
23
24    // --- Read third line ---
25    std::getline(infile, line);
26    std::stringstream(line) >> Lx;
27
28    // --- Read fourth line ---
29    std::getline(infile, line);
30    std::stringstream(line) >> Ly;
```

```

31
32  infile.close();
33
34  // display inputs
35  std::cout << "--- inputs --- " << std::endl;
36  std::cout << "Nx = " << Nx << std::endl;
37  std::cout << "Ny = " << Ny << std::endl;
38  std::cout << "Lx = " << Lx << std::endl;
39  std::cout << "Ly = " << Ly << std::endl;
40
41  return 0;
42 }

```

The key concept in File (4.3) appears on line 18:

```

18  std::stringstream(line) >> Nx;

```

This command performs two actions in one step:²

1. The expression `std::stringstream(line)` creates a temporary “*stream*” from the string stored in `line`. A stream behaves like a sequence of characters that you can extract data from, similar to reading input from the keyboard using `cin`.
2. The extraction operator `>> Nx` reads characters from the stream and converts them to the type of the variable `Nx` (here, an `int`). It starts from the beginning of the string and stops when it encounters a character that cannot be part of an integer. Because each line of the input file is formatted as a number followed by a semicolon, this effectively extracts the numeric value at the start of the line. Any text after the semicolon is ignored, allowing descriptive comments to be included on the same line.
3. Note that for this operation to work correctly, each line of the input file must begin with a numeric value. Any leading characters that are not part of the number will prevent the extraction operator from reading the value properly.

Compiling and running File (4.3) produces the terminal output below:

```

--- inputs ---
Nx = 16
Ny = 32
Lx = 2.1
Ly = 3.2

```

²This syntax is non-standard C++, but works correctly on most common compilers, including GCC and Clang.

4.3 Functions

The nuclear fuel rod described in Section 4.1 requires the long input file below:

```
1 16; Nx (number of cells in x)
2 32; Ny (number of cells in y)
3 2.1; Lx (domain length in x)
4 3.2; Ly (domain length in y)
5 0.5; R1 (radius of fuel core)
6 0.7; R2 (radius of cladding)
7 1.1; k1 (thermal conductivity of fuel core)
8 1.2; k2 (thermal conductivity of cladding)
9 1.3; k3 (thermal conductivity of coolant)
10 0.9; a (minimum volumetric heat generation)
11 2.3; b (maximum variation in heat generation)
12 10.0; Tc (coolant temperature)
```

If we use the approach from the previous section, we would need to repeat the steps of reading a full line and extracting the numeric value for each of the 12 variables above. To simplify the process, we can define a dedicated *function* that combines the steps of reading a line and extracting the data.

4.3.1 A basic C++ function

A C++ function can optionally take arguments, perform operations, and optionally return a value. Though C++ functions can take several forms, the simplest syntax is that below:

```
return_type functionName(type1 arg1, type2 arg2)
{
    // function body

    return r_value;
}
```

For a simple example, the function below takes two integers and returns their sum:

```
int addInt(int a, int b)
{
    int sum = a + b;
    return sum;
}
```

File 4.4 below demonstrates how we can use functions to simplify the syntax used to read data from files.

File 4.4: Defining functions that read data from a file.

```
1 #include <fstream>    // needed for reading files
2 #include <iostream>   // needed for cout
3 #include <sstream>    // needed for stringstream
4 #include <string>     // needed for string
5
```



```

6 // function to read an integer from a line
7 int readInt(std::ifstream &infile)
8 {
9     int value;
10    std::string line;
11    std::getline(infile, line);
12    std::stringstream(line) >> value;
13    return value;
14 }
15
16 // function to read a double from a line
17 double readDouble(std::ifstream &infile)
18 {
19     double value;
20     std::string line;
21     std::getline(infile, line);
22     std::stringstream(line) >> value;
23     return value;
24 }
25
26 int main()
27 {
28     int Nx, Ny;
29     double Lx, Ly;
30
31     // Read values from file
32     std::ifstream infile;
33     infile.open("inputs.inp");
34
35     Nx = readInt(infile);
36     Ny = readInt(infile);
37     Lx = readDouble(infile);
38     Ly = readDouble(infile);
39
40     infile.close();
41
42     // Display the inputs
43     std::cout << "--- inputs ---\n";
44     std::cout << "Nx = " << Nx << "\n";
45     std::cout << "Ny = " << Ny << "\n";
46     std::cout << "Lx = " << Lx << "\n";
47     std::cout << "Ly = " << Ly << "\n";
48
49     return 0;
50 }

```

Let's walk through the new concepts introduced in [File 4.4](#):

- **Header files:** Because the four headers are included at the top of the file, on lines 1–4,

they are accessible to all functions in the file: `readInt`, `readDouble`, and `main`.

- **Function definitions before main:** Before a function can be used in `main`, the compiler must know the function's argument types and return type. One way to ensure this is to define the functions before they are used in `main`. File 4.4 follows this approach, defining `readInt` and `readDouble` on lines 7–24, before entering `main` on line 26.
- **Separate functions for int and double:** Because our procedure for reading numeric data from a file depends on whether the data is an `int` or `double`, we define the function `readInt` for reading integers, and `readDouble` for reading doubles.
- **Opening the file in main:** Note that the file `inputs.inp` is opened only once, within `main` on line 33. We then pass the variable `infile` to the functions. This is necessary because if we opened the file within the functions `readInt` and `readDouble`, the functions would start reading from the top of the file each time. Instead, by opening the file once in `main`, we maintain a single reading position that advances as each line is read.
- **Passing by reference (&).** Unfortunately, passing an open file to a function is slightly more complicated than passing a simple `double` or `int`. Specifically, you may have noticed that on line 7, the argument `infile` has an ampersand in front of it:

```
7 int readInt(std::ifstream &infile)
```

We will explain the role of this ampersand symbol in more detail in Chapter 5. For now, it is sufficient to know that without the `&`, C++ passes a copy of `infile` to the function rather than the original. As a result, each function call would start reading from the top of the file, rather than continuing from where the previous call left off.

4.3.2 Function declarations (prototypes)

In the previous example, the functions `readInt` and `readDouble` were defined before `main`. An alternative approach is to *declare* a function first, then provide its full definition later in the file. This is done by providing the function's *declaration*, or *prototype*, above `main`. For example, we can modify File 4.4 as shown below.

```
#include <fstream>    // needed for reading files
#include <iostream>    // needed for cout
#include <sstream>     // needed for stringstream
#include <string>      // needed for string

// function prototypes
int readInt(std::ifstream &infile);
double readDouble(std::ifstream &infile);

int main()
{
    ...
    ...
}
```

```
// --- function definitions ---

int readInt(std::ifstream &infile)
{
    int value;
    std::string line;
    std::getline(infile, line);
    std::stringstream(line) >> value;
    return value;
}

double readDouble(std::ifstream &infile)
{
    double value;
    std::string line;
    std::getline(infile, line);
    std::stringstream(line) >> value;
    return value;
}
```

This separation of declaration and definition allows for more flexible code organization, especially in larger projects, and is a common C++ practice.

4.4 Homework

Problem 1: Variable thermal conductivity

In this problem, we extend the code developed in Chapter 3 to accept a variable thermal conductivity. We then verify the code using the manufactured solution below,

$$c_m = \cos(x) \cos(y), \quad k_m(x, y) = 1 + 0.5 \sin(x) \sin(y). \quad (4.4)$$

To determine the required source term $f(x, y)$, we substitute these expressions into the steady diffusion PDE:

$$-\nabla \cdot [k_m(x, y) \nabla c_m(x, y)] = q(x, y), \quad (4.5)$$

producing

$$q(x, y) = 2 \cos(x) \cos(y) [1 + \sin(x) \sin(y)]. \quad (4.6)$$

Perform the following steps to verify your solver:

- (a) Write a C++ function that takes the coordinates of a point (x, y) and returns the test conductivity $k_m(x, y)$.
- (b) Using the function from part (a), modify the construction of your matrix $\mathbf{A}$ to apply the variable conductivity $k(x, y)$ given in Eq. (4.4). For this, evaluate the conductivity directly at the face centroids. This avoids the need for interpolation and is sufficient for this introductory code.

- (c) Modify the source term in the right-hand-side vector $\mathbf{b}$ of the linear system.
- (d) Produce an error plot that verifies the second-order accuracy of your modified code.

Problem 2: Simulating the fuel rod

Now perform the following steps to simulate the nuclear fuel rod:

- (a) write a C++ function that returns the thermal conductivity of the fuel, cladding or coolant at a given point (x, y) in the domain. For that, the function can compute the distance r from the point (x, y) to the center of the rod,

$$r = \sqrt{(x - x_c)^2 + (y - y_c)^2}, \quad (4.7)$$

It can then output k using the rule

$$k(x, y) = \begin{cases} k_1, & r < R_1 \quad (\text{fuel core}), \\ k_2, & R_1 \leq r < R_2 \quad (\text{cladding}), \\ k_3, & r \geq R_2 \quad (\text{coolant}). \end{cases} \quad (4.8)$$

One convenient way to implement this logic is using an *if-else if-else* construction in C++, shown below

```

1    double k = k3;
2    if (r < R1)
3    {
4        k = k1;
5    }
6    else if (r < R2)
7    {
8        k = k2;
9    }
```

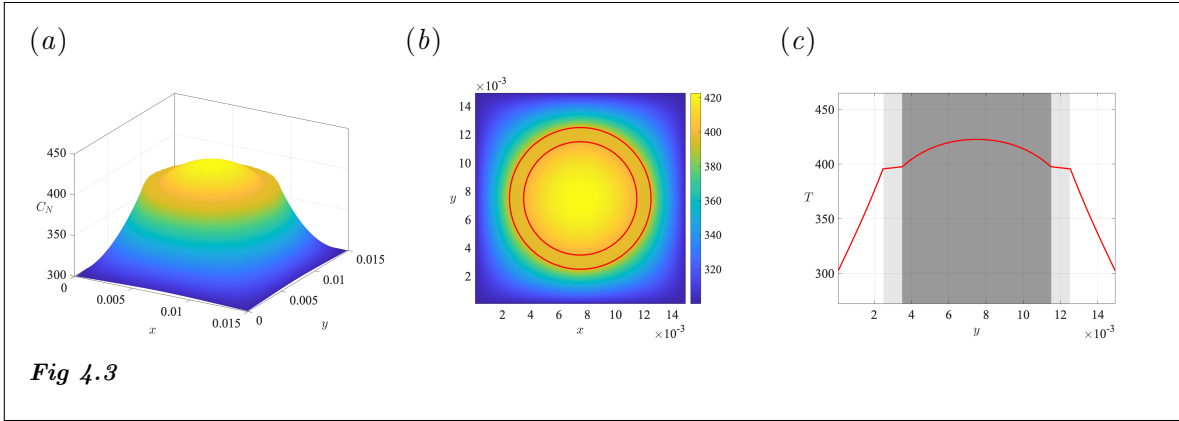
- (b) Modify the code that builds $\mathbf{b}$ to apply the source $s(x, y)$ in the fuel core

$$s(x, y) = a \left[1 + b \left(\frac{r}{R_1} \right)^2 \right]. \quad (4.9)$$

- (c) Modify the code that builds $\mathbf{b}$ to apply the temperature T_c on the boundaries
- (d) Run your code using the inputs below:

```

1    100;    Nx (number of cells in x)
2    100;    Ny (number of cells in y)
3    0.015;  Lx (domain length in x, m)
4    0.015;  Ly (domain length in y, m)
5    0.004;  R1 (radius of fuel core, m)
6    0.005;  R2 (radius of cladding, m)
7    2.0;    k1 (thermal conductivity of fuel core, W/m/K)
8    15.0;   k2 (thermal conductivity of cladding, W/m/K)
9    0.6;    k3 (thermal conductivity of coolant, W/m/K)
```



```

10 1.0e7; a (minimum volumetric heat generation W/m$^3$)
11 1.0; b (maximum volumetric heat generation)
12 300.0; Tc (coolant boundary temperature, K)

```

(e) Write a post-processing routine (using Python or Matlab) that reads the results and generates the three plots below. For comparison, Fig. 4.3 shows the expected results.

- (i) a 3-D surface plot of $T(x, y)$,
- (ii) a 2D color-plot of $T(x, y)$ with a colorbar and two superimposed circles showing the $r = R_1$ and $r = R_2$.
- (iii) a plot showing how the temperature varies along a vertical line $0 \leq y \leq L_y$ passing through the center of the fuel rod $x = x_c$.

Problem 3: Mesh independence study

- (a) Modify your C++ code or post-processing routine to compute the maximum temperature T_{max} in the fuel core, and the total heat transfer rate $\dot{Q}$ (per unit depth) generated by the fuel core.
- (b) Using the inputs given in Problem 2(d), run your code for the resolutions $N_x = N_y = N$, with $N = 4, 8, 16, 32, 64, 128, 256, 512$. For each resolution, record T_{max} and $\dot{Q}$.
- (c) Use your results to compute the observed order-of-accuracy p , the approximate relative error e_a^{21} , the extrapolated estimate ϕ_{ext}^{21} , the extrapolated relative error e_{ext}^{21} , and the grid convergence index GCI^{21} , for both T_{max} and $\dot{Q}$.
- (d) You should find that the observed order p does not settle cleanly near the expected second-order accuracy of the finite-volume method, and that T_{max} and $\dot{Q}$ vary non-monotonically with N rather than converging smoothly. Discuss why the discontinuous thermal conductivity at $r = R_1$ and $r = R_2$ might explain this behavior.